Todolist — Archetype Exploration

March 2026

Eight solution directions for an intelligent task system that replaces Todoist. Informed by landscape research into Taskwarrior, MCP task servers, and AI scheduling tools.

VISION **A task system that thinks alongside me — surfacing what matters most each day.**

Landscape over directives | Frictionless capture, rich context later | Deep integration over standalone

1 Custom CLI

Build from scratch in TS/Python with SQLite. Borrow Taskwarrior concepts — dependencies, hooks, JSON export — but purpose-built for your workflow.

Capture · Relationships · Landscape

More upfront work, but full control. No legacy baggage.

2 Taskwarrior Fork

Use actual Taskwarrior as task engine. Wrap with MCP server and AI layer for /morning skill. Keep its battle-tested CLI.

Capture · Relationships · AI Priorities

Inherits C++ complexity. Hard to customize data model.

3 MCP Server Only

The system IS an MCP server. Claude reads, writes, and reasons about tasks directly. No standalone UI — all interaction through AI sessions.

AI Priorities · Reflect+Reshape · Capture

Great integration, but can't capture without an AI session.

4 CLI + MCP Hybrid

CLI for human interaction (capture, browse). MCP server for AI interaction (/morning reads/writes). Same SQLite, two interfaces.

Capture · AI Priorities · Relationships

Two interfaces to maintain. But each is focused.

5 Morning Brew Ext

Extend Morning Brew directly. Add task management into the daily HTML. Tasks in Supabase. Reflection becomes reshaping interface.

Reflect+Reshape · Landscape · Capture

Deeply integrated but tightly coupled. No CLI capture.

6 Claude Code Skill

A /tasks skill living inside Claude Code. Tasks in markdown or SQLite. Capture via /tasks add. /morning reads from the same store.

Capture · AI Priorities · Reflect+Reshape

No mobile web. No browsing outside Claude Code.

7 Web App + API

Supabase-backed web app reusing Morning Brew's instance. REST API for /morning. Mobile-friendly UI for browsing and reshaping.

Landscape · Reflect+Reshape · Relationships

Most infrastructure. But gives you a real app on your phone.

8 CLI + MCP + Web

CLI for capture, MCP for AI integration, thin web view on Vercel for mobile. Three surfaces, one brain. Composable by design.

All 5 JTBDs

Most pieces. But each is small, focused, composable.

JTBD Coverage Matrix

How well each direction serves the five "Now" jobs from the strategy.

Direction	Capture Instantly	Full Landscape	Reflect + Reshape	AI Priorities	Relationships
1. Custom CLI	Strong	Partial	Weak	Weak	Strong
2. Taskwarrior Fork	Strong	Partial	Weak	Partial	Strong
3. MCP Server Only	Weak	Partial	Strong	Strong	Partial
4. CLI + MCP Hybrid	Strong	Partial	Strong	Strong	Strong
5. Morning Brew Extension	Weak	Strong	Strong	Partial	Partial
6. Claude Code Skill	Strong	Partial	Partial	Strong	Partial
7. Web App + API	Partial	Strong	Strong	Partial	Strong
8. CLI + MCP + Web	Strong	Strong	Strong	Strong	Strong

● Strong — core interaction directly serves this job ● Partial — requires adaptation ● Weak — significant stretch

KEY INSIGHT

Only two directions cover all 5 jobs

Direction 4 (CLI + MCP) and Direction 8 (CLI + MCP + Web) are the only architectures that naturally serve all five Now JTBDs. Every other direction has at least one Weak gap. The difference: #8 adds a mobile web surface for landscape browsing.

LANDSCAPE GAP

No existing tool bridges capture and reflection

Taskwarrior has the right data model (CLI + SQLite + dependencies) but no AI layer. MCP task servers exist but are agent-facing, not human-facing. The personal + self-owned + CLI + AI space is wide open.

FORK VS. BUILD

Taskwarrior's concepts, not its code

COMPOSABILITY

Three small surfaces beat one big app

Taskwarrior's architecture pattern (CLI, JSON data, dependencies, hooks) is gold. But its C++ codebase makes customization hard — especially adding reflection links, AI reasoning, and MCP integration. Build from scratch, borrowing concepts.

The strategy principle "deep integration, composable pieces" points toward direction 8. A CLI, an MCP server, and a thin web view — each independently improvable, all sharing one SQLite/Supabase brain.